西安交通大学 XIAN JIAOTONG UNIVERSITY

现代密码学 COMP401227

第 5 章：消息认证和安全哈希函数

赵俊舟

西安交通大学网安学院

junzhou.zhao@xjtu.edu.cn

2026 年 4 月 23 日

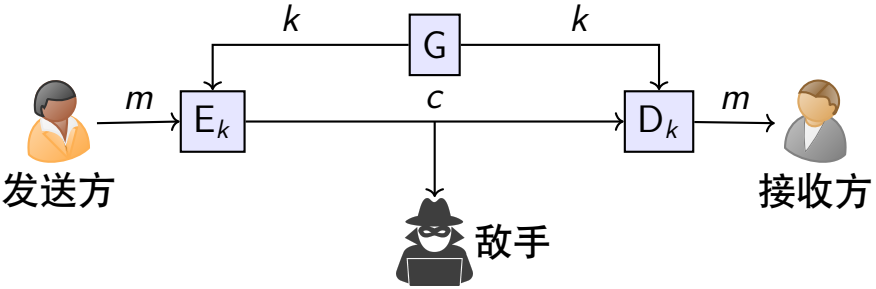
目录

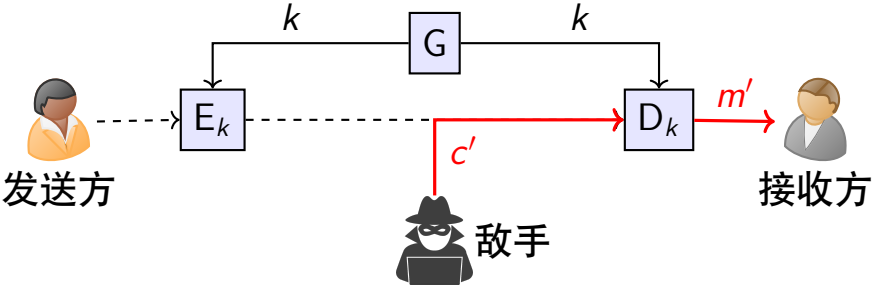
1. 消息认证
2. 密码学哈希函数
3. 安全哈希算法
4. HMAC 算法

目录

1 消息认证
2 密码学哈希函数
3 安全哈希算法
4 HMAC 算法

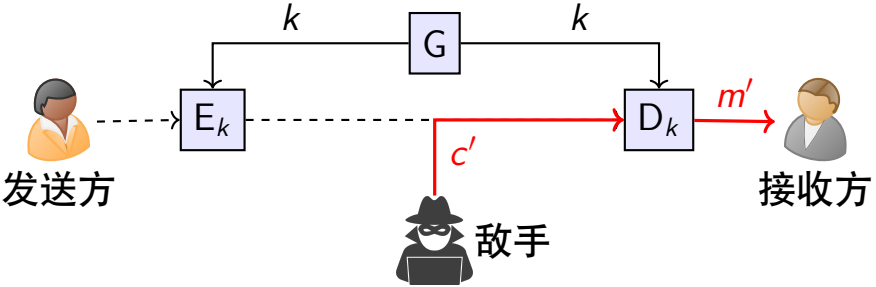
被动作恶与主动作恶





消息保密与消息认证

- 对称加密可以保证消息的保密传输，但无法防止敌手主动作恶：伪造消息和篡改消息。
- 伪造消息：银行收到一条伪造的转账请求，要求从账户 A 转账 1 万元到账户 B。
- 篡改消息：账户 A 要求银行转账 1 万元到账户 B，消息被敌手修改为转账 10 万元到账户 C。



网络通信环境中的攻击

- **泄密**：将消息透露给没有密钥的第三方
- **传输分析**：分析双方通信模式

} 消息保密

- **消息伪装**：欺诈源向网络中插入一条消息
- **身份伪装**：发送方伪装为其他身份
- **内容篡改**：对消息内容的修改
- **顺序篡改**：对消息顺序的修改
- **计时篡改**：对消息的延时和重放

} 消息认证

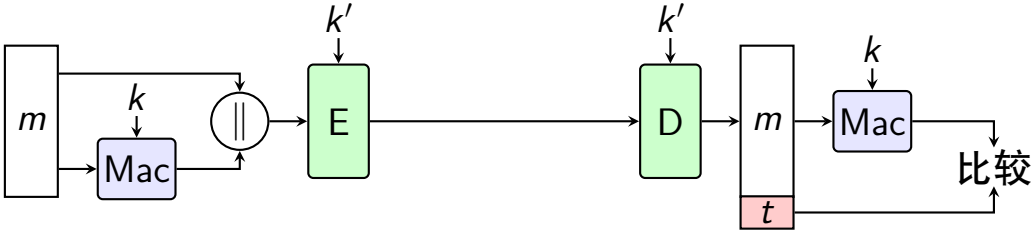
- **信源抵赖**：发送方否认发送过某消息
- **信宿抵赖**：接收方否认接收过某消息

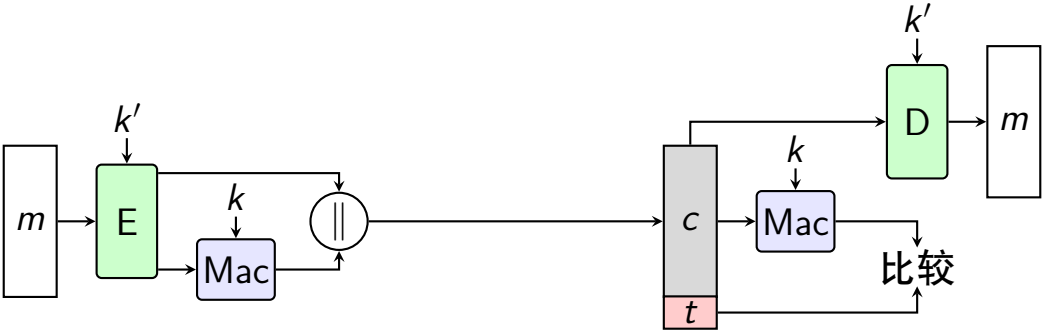
} 数字签名 + 其他手段

消息认证码（Message Authentication Code）

- 消息认证的作用是实现通信双方对消息完整性的验证，防止敌手篡改或伪造消息。
- 通信双方都持有密钥 k；
- 发送方在发送消息 m 前，计算消息 m 的消息认证码，即 $t = \mathrm{Mac}_k(m)$，然后将 (m, t) 作为报文整体发送给接收方；
- 接收方收到 (m, t) 后，使用验证算法 Vrfy 对报文进行验证。

消息认证码的其他工作模式





消息认证码的定义

定义 (消息认证码，Message Authentication Code, MAC)

消息认证码包括三个算法 (Gen, Mac, Vrfy)：

- 密钥生成函数 Gen：产生满足安全性要求的密钥 k；
- 消息认证码生成函数 Mac：输入密钥 k 和消息 $m \in \{0,1\}^*$，输出消息认证码 $t \leftarrow \mathrm{Mac}_k(m)$；
- 验证函数 Vrfy：输入密钥 k，消息 m 和消息认证码 t，输出比特 $b = \mathrm{Vrfy}_k(m, t)$。

- 作用：验证消息来源的真实性和完整性。
- 正确性要求：$\mathrm{Vrfy}_k(m, \mathrm{Mac}_k(m)) = 1$。

消息认证码的特点和要求

- 消息认证码生成函数 Mac 是一种多对一函数，类似于加密，但不要求可逆。
- 消息认证码生成函数的定义域由任意长的消息组成，值域由所有可能的消息认证码组成。
- 若攻击者已知 m 和 t，则构造满足 $\mathrm{Mac}_k(m') = t$ 的消息 m' 在计算上不可行。
- $\mathrm{Mac}_k(m)$ 是均匀分布的，对任何随机选择的消息 m 和 m'，
$$\Pr(\mathrm{Mac}_k(m') = \mathrm{Mac}_k(m)) = 2^{-n}$$
其中 n 是消息认证码的位数。

实际应用：肩窥攻击与双因素认证

- 肩窥攻击（Shoulder Surfing Attack）

- 双因素认证（Two-Factor Authentication）

- A user logs into a service using something they know (e.g., a password) and something they have (e.g., a phone).
- Timed One-Time Password (TOTP): $t = \mathrm{Mac}_k(s)$ where s is current time and rounded into 30 seconds.

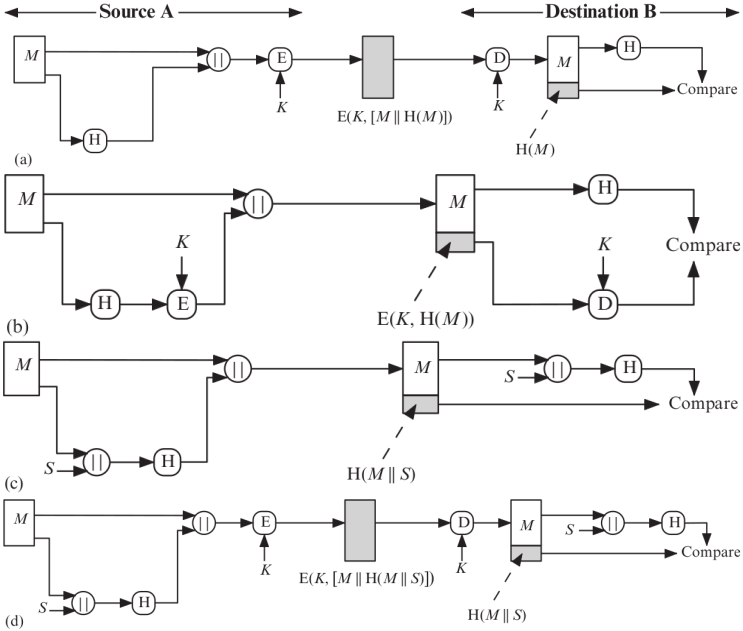
目录

1. 消息认证
2. 密码学哈希函数
3. 安全哈希算法
4. HMAC 算法

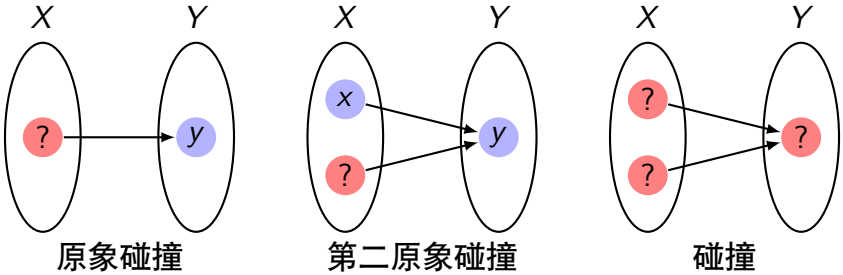
密码学哈希函数

- 一个密码学哈希函数以变长的消息 m 作为输入，产生定长的哈希码 $H(m)$ 作为输出，哈希码 $H(m)$ 亦称作报文摘要。
- 密码学哈希函数可以应用于任意大小的数据块。
- 密码学哈希函数产生固定长度的输出。
- 哈希码是消息所有比特的函数值，具有差错检测能力，消息任意一比特的改变都将引起哈希码的改变。
- 因此哈希码可以当作消息认证码来使用。

哈希函数用于消息认证


Source A
Destination B
M
||
E
K
E(K, [M || H(M)])
D
K
M
H
Compare
H(M)
H
(a)
M
||
M
H
K
Compare
H
E
E(K, H(M))
K
D
(b)
M
||
M
S
||
H
Compare
S
||
H
H(M || S)
(c)
M
||
E
K
E(K, [M || H(M || S)])
D
K
M
S
||
H
Compare
S
||
H
H(M || S)
(d)

抗碰撞性要求



- 抗原象碰撞（Preimage Resistance）：给定 $y = H(x)$，很难找到 x'，使 $H(x') = y$，即单向性。 弱
- 抗第二原象碰撞（Second-Preimage Resistance）：给定 x，很难找到 $x' \neq x$，使 $H(x) = H(x')$，即抗弱碰撞性。 较强
- 抗碰撞（Collision Resistance）：很难找到两个输入 $x \neq x'$，使 $H(x) = H(x')$，即抗强碰撞性。 强

对哈希函数的原像攻击

- **原像攻击**：攻击者对给定的哈希值 h，试图找到满足 $H(x) = h$ 的消息 x。
- **穷举攻击法**：攻击者随机选择 x，计算其哈希值，直到碰撞出现。

原像攻击的数学描述

若一个函数可能有 n 个值，且已知一个值 $H(x)$，任选 k 个任意数作为输入，则 k 至少为多大才能保证至少找到一个输入值 y 且 $H(y) = H(x)$ 的概率大于 $1/2$？

当 $k > n/2$ 时，碰撞概率将超过 $1/2$。

对哈希函数的原像攻击

- 对于任意 y，满足 $H(y) = H(x)$ 的概率是 $1/n$。
- 则 k 个任意输入，至少有一个满足 $H(y) = H(x)$ 的概率是 $1-(1-1/n)^k$，根据二项式定理
$$(1-a)^k = 1 - ka + \frac{k(k-1)}{2!}a^2 - \frac{k(k-1)(k-2)}{3!}a^3 \ldots$$
当 $a \to 0$ 时，$(1-a)^k \approx 1 - ka$。
- 所以，至少找到一个 y 满足 $H(y) = H(x)$ 的概率几乎等于 $1-(1-k/n) = k/n$。当 $k > n/2$ 时，这个概率将超过 $1/2$。

Robin 攻击方法

64 位的哈希函数，有 2^{64} 种组合，攻击者需要尝试 2^{63} 个消息，就有可能获得超过 $1/2$ 的成功机会产生某个特定哈希码上的碰撞。

对哈希函数的碰撞攻击

- 碰撞攻击：攻击者试图找到两个消息 x 和 y，满足 $H(x) = H(y)$，也称为生日攻击（Birthday Attack）。
- 可以证明，碰撞攻击的穷举规模要比原像攻击的穷举规模小很多。
- 生日悖论（Birthday Paradox）：
  - 23 个人中，存在两个人生日相同的概率大于 50%；
  - 50 个人中，存在两个人生日相同的概率大于 96%。
- Yuval 提出用生日悖论对哈希函数进行碰撞攻击，证明 64 位哈希函数只需要 2^{32} 次运算就有可能获得超过 1/2 的成功机会。

生日攻击的数学背景

生日攻击的数学描述

从编号为 1 到 n 的 n 个球中有放回的随机取出 k 个球，当 k 为多大时可以保证取出的 k 个球有重复的概率大于 $1/2$？

- 有放回随机取 $k < n$ 个球，球的编号互不相同的概率为

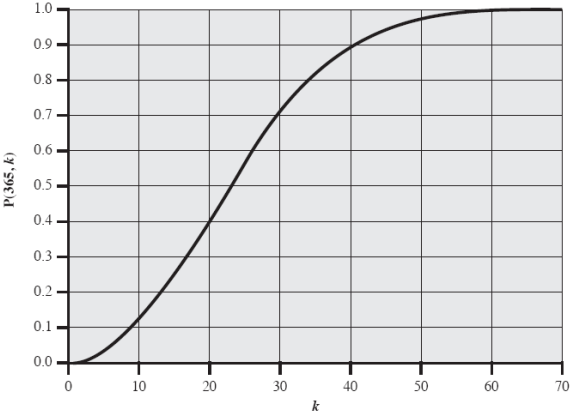
$$\frac{n(n-1)\cdots(n-k+1)}{n^k} = \prod_{i=0}^{k-1}(1-\frac{i}{n}) \leq \prod_{i=0}^{k-1} e^{-i/n} = e^{-\frac{k(k-1)}{2n}}$$

  利用了 $1-x \leq e^{-x}, x \in [0,1]$。

- 则至少两个球编号相同的概率为 $1-e^{-\frac{k(k-1)}{2n}}$
- 希望上述概率不小于 0.5，得出 $k^2 > k(k-1) \geq 2n\ln 2$，所以 $k > 1.17\sqrt{n}$。

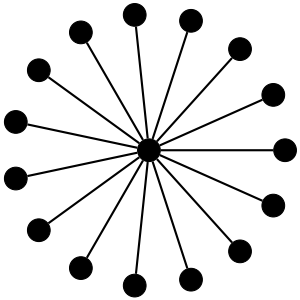
生日悖论

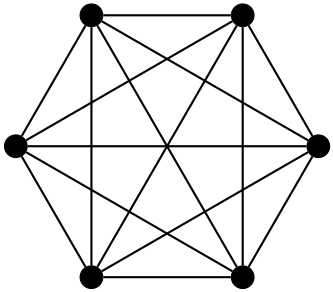
- 在 23 个人中，考虑某个人的特定生日，在剩下的 22 个人中找到相同生日的概率只有 $22/365$；
- 但是若只考虑同一天出生，那么 23 个人会产生 $\binom{23}{2} = 253$ 种不同的组合，所以找到同生日的概率 $253/365 > 1/2$。



从图论角度理解生日悖论

原象攻击





生日攻击

- 一条边表示一次哈希碰撞，边数表示碰撞的机会数。
- 原象攻击中，图为星形，要获得 n 次碰撞机会，得有 n 个节点（即需要进行 n 次哈希以获得 n 次碰撞机会）。
- 生日攻击中，图为完全图，要获得 n 次碰撞机会，只需 $\sqrt{n}$ 个节点（即只需进行 $\sqrt{n}$ 次哈希就能获得 n 次碰撞机会）。

目录

1. 消息认证
2. 密码学哈希函数
3. 安全哈希算法
   - SHA 标准
   - SHA 应用
4. HMAC 算法

目录

1 消息认证

2 密码学哈希函数

3 安全哈希算法
- SHA 标准
- SHA 应用

4 HMAC 算法

安全哈希算法（SHA）

- SHA 是由美国标准与技术协会 NIST 设计，并于 1993 作为联邦标准 FIPS 180 发布，1995 年修订为 FIPS 180-1，即 SHA-1。
- SHA 算法建立在 MD4 算法之上，基本框架与 MD4 类似。
- SHA-1 产生 160 位哈希值，以后的修订版分别为 SHA-256，SHA-384，SHA-512，与 SHA-1 有相同的基础结构。
- 2005 年 NIST 宣布了逐步废弃 SHA-1 的意图。一周后，王小云在 Crypto 2005 会议上发表 Finding Collisions in the Full SHA-1，宣布了一种攻击方法，用 2^{69} 次操作找到两个独立的消息具有相同哈希值（此前认为需要 2^{80} 次操作）。这一结果加速了向 SHA 其他版本的过渡。
- 2022 年 12 月，NIST 建议 2030 年底前淘汰 SHA-1。

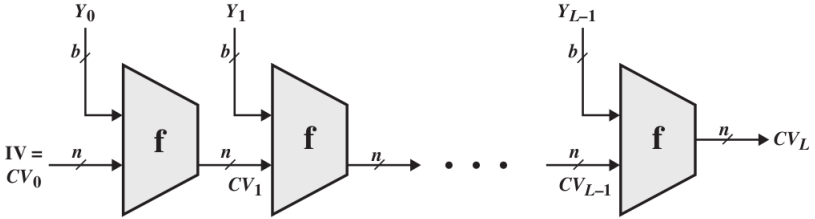
SHA 参数比较

Algorithm	Message Size	Block Size	Word Size	Message Digest Size
SHA-1	$< 2^{64}$	512	32	160
SHA-224	$< 2^{64}$	512	32	224
SHA-256	$< 2^{64}$	512	32	256
SHA-384	$< 2^{128}$	1024	64	384
SHA-512	$< 2^{128}$	1024	64	512
SHA-512/224	$< 2^{128}$	1024	64	224
SHA-512/256	$< 2^{128}$	1024	64	256

Note: All sizes are measured in bits.

密码学安全哈希函数的结构

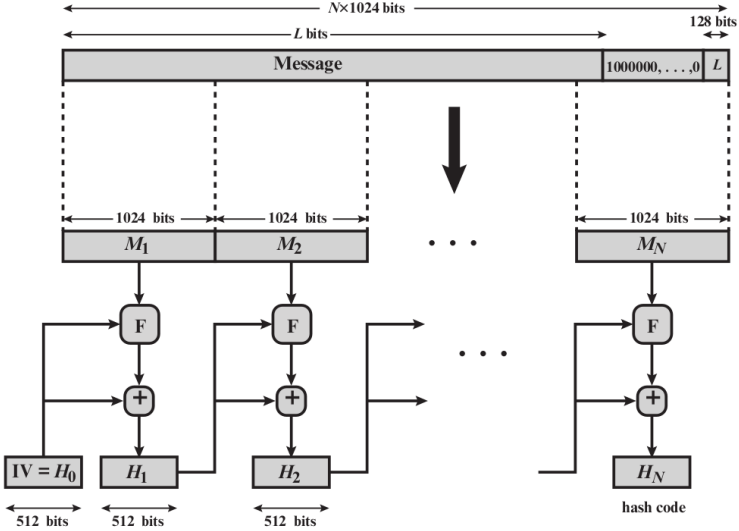
- 包括 SHA 在内的绝大多数密码学安全哈希函数都是基于迭代哈希函数结构



- 将输入消息分为 L 个固定长度的分组，每组 b 位。
- 最后一个分组不足 b 位时要填充，并且最后一个分组包含消息总长度。
- 重复使用压缩函数 f 对前一步得到的 n 位分组（称为链接变量 CV）和当前 b 位输入分组进行运算，得到一个 n 位分组。

SHA-512 总体结构

算法的输入是最大长度小于 2^{128} 位的消息，输出是 512 位的消息摘要，输入消息以 1024 位的分组为单位处理。



SHA-512 算法步骤

1. **附加填充位**：使消息长度 $\equiv 896 \pmod{1024}$，填充位数在 1 到 1024 之间，由 1 和后续的 0 组成。
2. **附加长度**：在消息后附加 128 位的块，将其看作 128 位无符号整数，代表填充前消息的长度。
3. **初始化哈希缓冲区**：8 个 64 位寄存器（8 个字）

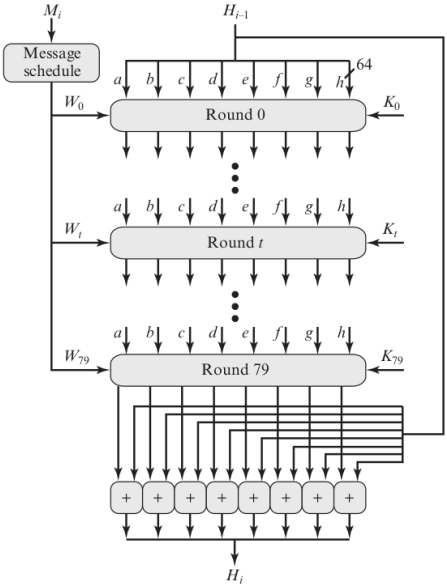
$$
\begin{aligned}
a &= \texttt{6A09E667F3BCC908} & e &= \texttt{510E527FADE682D1} \\
b &= \texttt{BB67AE8584CAA73B} & f &= \texttt{9B05688C2B3E6C1F} \\
c &= \texttt{3C6EF372FE94F82B} & g &= \texttt{1F83D9ABFB41BD6B} \\
d &= \texttt{A54FF53AF1D336F1} & h &= \texttt{5BE0CD19137E2179}
\end{aligned}
$$

4. **以 1024 比特分组为单位处理消息**：核心是 80 轮运算，每一轮都把 512 位缓冲区的值 *abcdefg* 作为输入并更新。
5. **输出**：所有 N 个 1024 比特分组都处理完后从第 N 阶段输出的是 512 比特的消息摘要。

SHA-512 的运算

$$H_0 = \mathrm{IV}$$
$$H_i = \mathrm{SUM64}(H_{i-1}, abcdefgh_i)$$
$$\mathrm{MD} = H_N$$

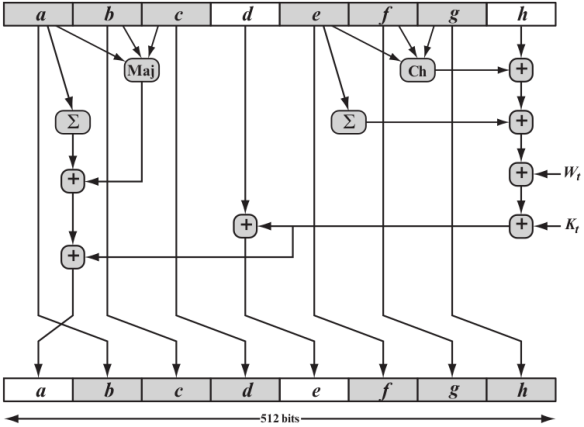
- IV：缓冲区初值。
- $abcdefgh_i$：第 i 个分组处理的输出
- N：消息中的分组数。
- SUM64：模 2^{64} 加。
- MD：最后的消息摘要。
- $K_t, t = 0, \ldots, 79$ 为轮常数，用来使每一轮的运算不同。
- $W_t, t = 0, \ldots, 79$ 由消息导出。



对单个 1024 位分组的处理

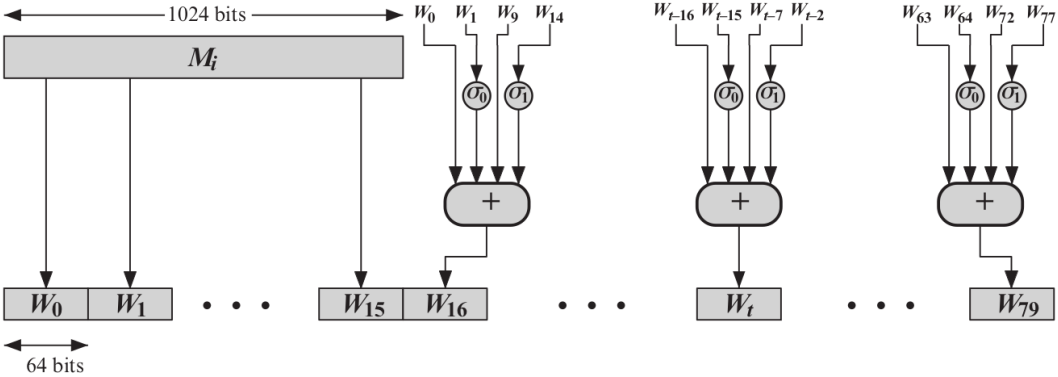
SHA-512 轮函数

- 输出 8 个字中的 6 个是简单的轮转置换（阴影部分）。
- 输出中只有 2 个字是通过替代置换产生的。
- Σ、Maj 和 Ch 为复杂二进制运算。



W_t 的导出

- 前 16 个 W_t 直接取自当前分组的 16 个字，
- 余下的 64 个 W_t 由前面的 4 个 W_t 推导得出，
- σ_0, σ_1 为复杂二进制运算。



SHA-512 的特点

- Hash 码的每一位都是全部输入位的函数
- 基本函数 F 多次复杂重复运算使得结果充分混淆
- 找到两个具有相同 Hash 码的消息的复杂度为 2^{256} 次操作
- 给定 Hash 码，寻找消息的复杂度为 2^{511} 次操作

目录

1. 消息认证
2. 密码学哈希函数
3. 安全哈希算法
   - SHA 标准
   - SHA 应用
4. HMAC 算法

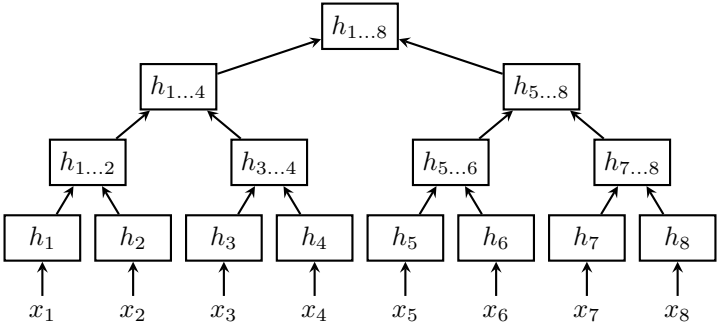
指纹和去重

- 如果 H 为抗碰撞哈希函数，则 $H(x)$ 可以作为输入 x 的唯一标识，或称为指纹。
- 病毒指纹：杀毒软件的病毒库记录了已知病毒的指纹，即对于一份病毒代码 x，其指纹为 $H(x)$。
- 去重：在云存储场景中，用户上传文件到云中。为了避免重复存储同样的文件，可以通过指纹比对确定是否已经存储过该文件。
- P2P 文件共享：P2P 网络中，peer 节点可以广播其存储的文件哈希，使其他 peer 节点知道文件的存储位置。

Merkle 树

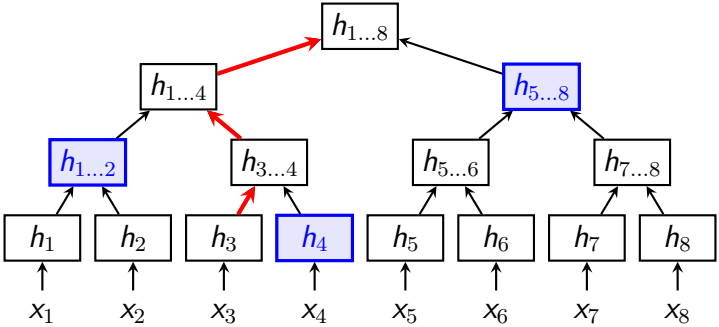
- 客户端上传一个文件 x 到服务器，之后从服务器下载相同的文件，客户端如何确保文件没有被篡改？
- 客户端上传文件 x 的同时，在本地保存文件的哈希值 $h = H(x)$。下载文件 x' 后，重新计算哈希值 $H(x')$，如果 $H(x') = h$，则认为文件未被篡改，即 $x' = x$。
- 推广到有 t 个文件 $x_1, \ldots, x_t$ 的情况。客户端上传这 t 个文件的同时，在本地保存哈希值 $h = H(x_1, \ldots, x_t)$。
- 客户端下载其中一个文件 x_i，如何保证该文件未被篡改？此时，客户端需下载其余 $t-1$ 个文件，然后重新计算其哈希，与 h 比较。
- 有没有更好的方法，能够减少客户端需要下载的数据量？

Merkle 树



- Ralph Merkle 最早提出用 Merkle 树解决该问题。
- Merkle 树的每个叶子节点 i 存储哈希值 $h_i = H(x_i)$，其他节点存储其子节点哈希值拼接后的哈希，例如 $h_{3...4} = H(h_3, h_4)$。
- 客户端在本地保存根节点存储的哈希值，例如 $h_{1...8}$。

Merkle 证明



- 当服务器返回 x_i 时，同时返回从根节点到叶节点 x_i 的路径上的所有节点的兄弟节点存储的哈希值，构成 Merkle 证明。
- 例如，返回 x_3 的同时，须返回 Merkle 证明 $\{h_4, h_{1\ldots2}, h_{5\ldots8}\}$。
- 客户端计算 $h'_{3\ldots4} = H(H(x_3), h_4), h'_{1\ldots4} = H(h_{1\ldots2}, h'_{3\ldots4})$ 和 $h'_{1\ldots8} = H(h'_{1\ldots4}, h_{5\ldots8})$，并与本地 $h_{1\ldots8}$ 比对实现验证。

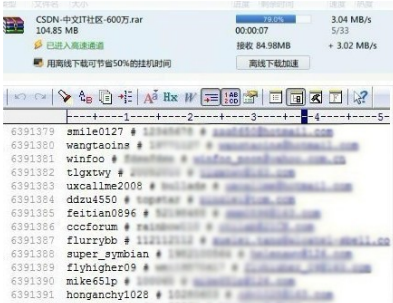
密码保护

- 考虑一个登录系统，用户输入用户名和密码，如果后台验证通过，则授权用户访问系统。
- 如果后台使用明文存储用户密码，则存在严重的安全风险。

CSDN被黑：600余万个明文的注册邮箱帐号和密码被黑客公开

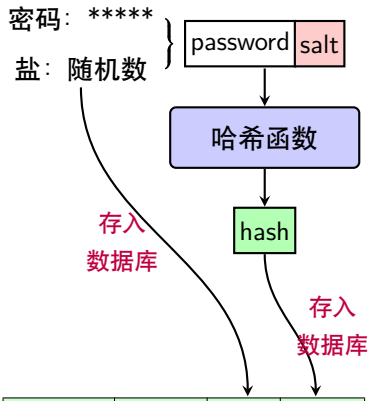
2011年12月21日 17:00

号称中国最大的开发者技术社区"中国软件开发联盟"（CSDN）数据库被黑了。360安全卫士发布消息说有600余万个明文的注册邮箱帐号和密码被黑客公开。



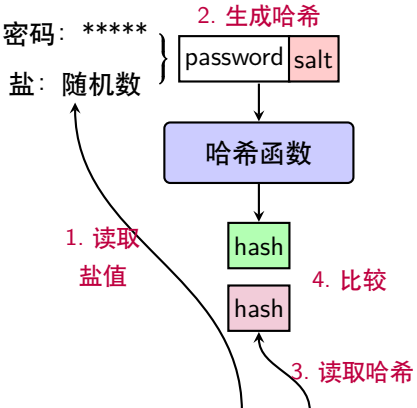
密码保护

存储密码



User ID	Name	Salt	Hash
123	Tom		
124	Jerry		

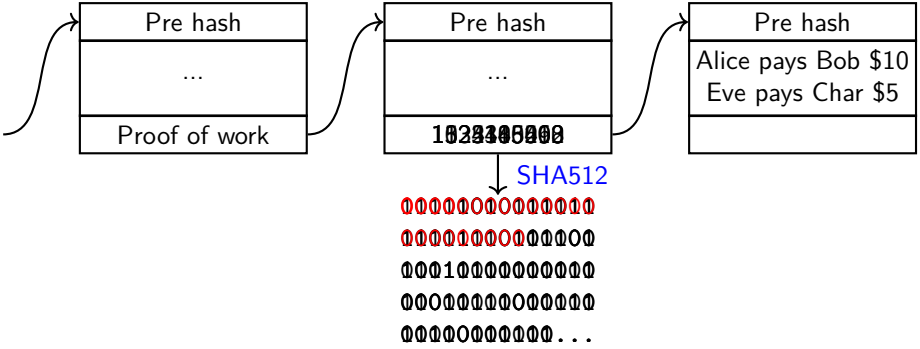
验证密码



User ID	Name	Salt	Hash
123	Tom		
124	Jerry		

区块链中的工作量证明

- 区块链是一群分散的客户端节点，并由所有参与者组成的分布式数据库，是对所有比特币交易历史的记录。
- 基于工作量证明机制（Proof of Work）实现将新的交易记录链到区块链上，形成一个新的区块。
- PoW 的过程即为挖矿，通过哈希函数实现。



目录

1. 消息认证
2. 密码学哈希函数
3. 安全哈希算法
4. HMAC 算法

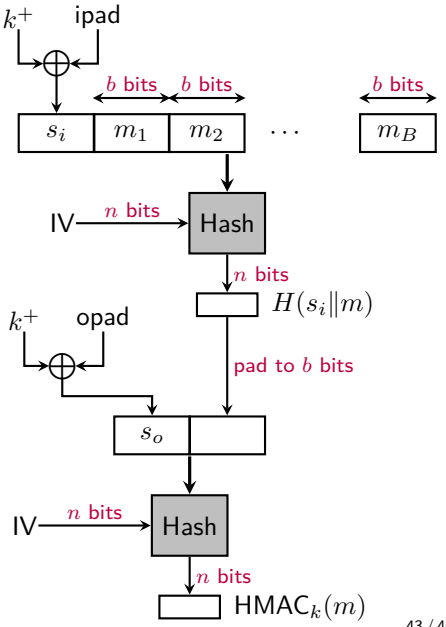
基于哈希函数的 MAC：HMAC

- 人们越来越感兴趣利用密码学哈希函数设计 MAC，因为：
  - 哈希函数软件执行速度比对称分组密码快。
  - 有许多密码学哈希函数代码库。
- HMAC（RFC2104）给出 HMAC 的设计目标是：
  - 不必修改而直接使用现有的哈希函数。
  - 用新哈希函数替代原来嵌入的哈希函数很容易。
  - 能保持哈希函数原有特性，不过份降低其性能。
  - 对密钥的使用和处理应较简单。
  - 如果已知嵌入的哈希函数的强度，则完全可知认证机制抗密码分析的强度。

基于哈希函数的 MAC：HMAC

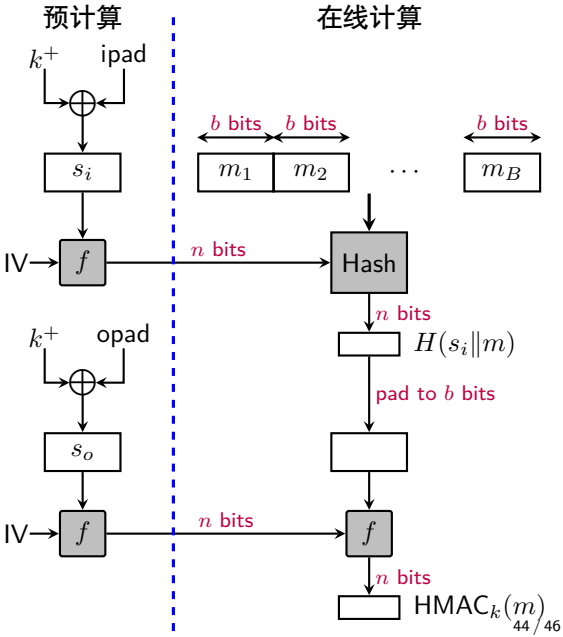
- 在 k 左边填充 0，得到 b 位的 k^+。
- k^+ 与 ipad 异或，产生 b 位分组 s_i。
- 将消息 m 附于 s_i 后，进行哈希。
- k^+ 与 opad 异或，得 b 位分组 s_o。
- 将步骤 3 中的哈希码附于 s_o 后。
- 哈希后得到最终消息认证码：

$$\mathrm{HMAC}_k(m) = H\big(s_o \| H(s_i \| m)\big)$$



通过预计算提高 HMAC 的计算效率

- 相比于消息哈希，HMAC 多执行了三次压缩函数 f。
- 对于长消息，HMAC 和哈希函数的执行时间大致相同。
- 对于短消息，可以通过预计算其中的两个压缩函数来提高效率。
- 这样在线计算只多执行了一次压缩函数。



HMAC 的安全性

- 任何建立在嵌入哈希函数基础上的 MAC，其安全性在某种程度上依赖于该哈希函数的强度。
- 攻击 HMAC 意味着：
  - 对密钥的穷举攻击。
  - 生日攻击。
- M. Bellar 等已经证明，如果攻击者已知若干（时间，消息-MAC）对，则成功攻破 HMAC 的概率等价于对嵌入哈希函数的下列攻击之一：
  - 即使 IV 是随机、秘密和未知的，攻击者也能够计算压缩函数的输出。
  - 即使 IV 是随机、秘密和未知的，攻击者也能够找到哈希函数中的碰撞。

小结

1. 消息认证
2. 密码学哈希函数
3. 安全哈希算法
4. HMAC 算法